SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

PROGRAMMING IN JAVA

Course Code: CSA0915A | Course Outcome: CO1

ASSIGNMENT 1

CAMPUS SMART-PARKING SYSTEM

Design, Implementation and Validation of an Object-Oriented Model

Tasks 1 to 12

Student Name:	Bhagya Shree S
Register Number:	192521338
Course:	Programming in Java (CSA09)
Assignment:	Assignment 1 – Campus Smart-Parking System (Tasks 1–12)
GitHub Repository:	https://github.com/bhagyashree3107/CSA09-192521338-Assignment1

1. Abstract

This report presents the analysis, design, implementation plan, validation strategy and design re-evaluation of a Campus Smart-Parking System, prepared for Assignment 1 of the course Programming in Java (Course Outcome CO1). The system models a multi-level university parking facility used by students, staff and visitors. Vehicles may be two-wheelers, cars or electric vehicles (EVs). On entry the system allocates a free slot, issues a ticket with a unique number and records the entry time. On exit it releases the slot and computes a fee that depends on the vehicle type and the duration of parking. Staff and students with a valid monthly pass park free, while visitors pay the full fee.

The report follows the twelve tasks of the assignment in order. Tasks 1 to 3 analyse the problem, choose data types and a jagged-array representation for floors and slots, and design the allocation and fee logic. Tasks 4 to 10 apply the object-oriented principles of Java: classes and constructors, static members, inheritance, overriding, polymorphism, abstraction, packages and encapsulation. Task 11 defines the test and evaluation strategy, and Task 12 re-evaluates the design against changed requirements (e-bikes, delivery vans, one-day visitor passes and a five-floor facility with unequal floors). The final decision is to modify, rather than redesign, the original model.

All results in this report are stated as expected results derived from the assignment brief and from hand calculation. Actual program outputs and screenshots are left as clearly marked placeholders, to be filled from the student's own execution recorded in the GitHub repository.

2. Introduction

Object-oriented programming allows a real-world problem to be represented as a set of cooperating objects, each responsible for its own data and behaviour. A parking facility is a good subject for this approach because it contains clearly identifiable entities (floors, slots, vehicles, tickets, users and passes), simple but important rules (a slot holds only one vehicle; a ticket number is never reused) and behaviour that differs between categories (the fee for a two-wheeler differs from the fee for a car).

This assignment asks the student to design, implement and validate such a model in Java, and to justify every design decision with evidence. Each task is followed by an alternative concept that has to be compared using Big-O time and space analysis. The purpose is to develop analytical skill, so that the reasons behind a design are understood as clearly as the design itself.

The report is organised as follows. Chapters 3 to 5 define the problem, objectives and project overview. Chapter 6 explains all twelve tasks in order. Chapters 7 and 8 describe the system design and the main algorithms. Chapters 9 to 11 cover testing, results and complexity. Chapters 12 to 15 present limitations, future enhancements, the conclusion and the repository reference.

3. Problem Statement

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles. EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may be as small as 1 floor of 20 slots, or may grow to 3 floors of 50 slots and 5 floors of 100 slots. The student must analyse the problem, design an object-oriented model, implement it in Java and validate it. The design must remain correct and easy to extend as the facility and its rules grow. Finally, in a design challenge, the university adds new vehicle categories and a new pricing rule, and the student must decide whether the original design should be retained, modified or redesigned.

4. Objectives of the Project

- To analyse the parking scenario and identify the objects, responsibilities, business rules and assumptions of the system (Task 1).
- To choose suitable Java data types and an array structure for floors and slots, and to avoid precision errors in fee calculation (Task 2).
- To design the control logic for slot allocation and fee computation, and to express it in pseudocode before coding (Task 3).

- To design classes, constructors, static members and an object life cycle that keep every object valid (Tasks 4 and 5).
- To build a vehicle inheritance hierarchy and apply overriding, polymorphism, abstract classes and interfaces (Tasks 6 to 9).
- To enforce encapsulation through packages, access modifiers and validated setters (Task 10).
- To implement, test and evaluate the complete system using normal, boundary and invalid cases and a hand-calculated scenario (Task 11).
- To re-evaluate the design against changed requirements and defend the decision with evidence (Task 12).
- To compare each design with an alternative concept using time and space complexity in Big-O notation.

5. Project Overview

5.1 The Campus Smart-Parking System

The Campus Smart-Parking System is a Java program that manages entry and exit of vehicles in a multi-floor campus parking facility. It keeps track of which slots are occupied, issues tickets, and computes fees. Its main purpose is to replace manual slot management with a consistent, rule-based process that cannot allocate one slot to two vehicles or bill a vehicle incorrectly.

5.2 Functionality

Table 1: Main functions of the system

Function	Description
Slot allocation	Searches floor by floor and slot by slot for the first free slot and places the vehicle there; reports failure when the facility is full.
Ticket issue	Creates a ticket with a unique number and records the entry time at the moment a slot is allocated.
Slot release	Frees a slot when a vehicle leaves; rejects an invalid or already-free position.
Fee computation	Computes the parking fee from the vehicle type and the parking duration, charging every started hour in full.
Pass handling	Treats students and staff with a valid monthly pass as fee-exempt, and charges visitors the full fee.
EV charging	Charges electricity used by an electric vehicle separately from the parking fee.

5.3 Assumptions Used in the Assignment

The assignment brief and its program listings fix the following values and rules, which are used throughout this report:

- Floors and slots are numbered from zero inside the program, and the program stores a vehicle number in each slot (null means free).
- The hourly rates in the supplied FeeDemo listing are Rs 10 per hour for a two-wheeler and Rs 30 per hour for a car. The EV charging rate is Rs 12 per kWh.
- Every started hour is charged in full, so billable hours are the ceiling of minutes divided by 60.
- Staff and students with a valid monthly pass park free; visitors pay the full fee.
- The facility size may vary from 1 floor of 20 slots to 5 floors of 100 slots, and in Task 12 each floor may have a different number of slots.

5.4 Additional Design Assumptions and Placeholders

The brief does not state some details. These are treated as the student's own design assumptions and are marked wherever they are used. Text inside square brackets, such as [Placeholder], marks a value or output that must be supplied from the student's own repository or execution.

- A daily maximum fee is mentioned as a possible slab in Task 3, but no amount is given. The amount is a placeholder: [DAILY_MAX – to be decided by the student].
- All times use one campus time zone and elapsed time is measured in whole minutes.
- Ticket numbers use the form T1, T2, T3 and so on (a design choice, not stated in the brief).
- Pseudocode is written in the ALGORITHM … END ALGORITHM style. If the SIMATS Engineering pseudocode standard requires a different notation, only the notation needs to be adjusted.
- The flat e-bike fee, the van tariff outside 9 AM – 5 PM and the fee behaviour of a one-day pass (Task 12) are not stated in the brief; the assumptions made are listed in Section 6.12 and should be confirmed with the course faculty.

6. Task-wise Explanation

This chapter explains the twelve tasks in the order of the assignment. For each task it gives the purpose, the Java concepts used, the implementation approach, the expected result and the alternative concept with its complexity analysis. Task titles follow the assignment brief. Where the brief asks for code, only the code required by the brief is given.

6.1 Task 1 – Understand the Problem and Identify the Objects

Purpose

This task analyses the scenario before any code is written. Its purpose is to identify the objects the program must model, the data that flows in and out, the rules every operation must respect, and the assumptions made about the facility.

Java Concepts Used

Classes and objects, fields and methods, abstract classes and interfaces (identified at this stage), and the choice between objects and parallel arrays.

Implementation Approach

Inputs and outputs. The system receives the vehicle number, vehicle type, user type (student, staff or visitor), pass status, entry and exit times and, for an EV, the energy consumed. It produces the allocated floor and slot, a unique ticket number, the entry and exit times, the billable duration, the fee, a slot-release confirmation and error messages.

Nouns and verbs. Nouns in the scenario – facility, floor, slot, vehicle, ticket, user, pass, fee, charger – become candidate classes or fields. Verbs – allocate, release, enter, exit, calculate fee, issue ticket, find ticket – become methods.

Table 2: Candidate classes, responsibilities and justification

Candidate class	Responsibility	Why it is needed
ParkingFacility	Holds floors and slots; performs allocate and release.	Occupancy is a property of the whole facility, not of a vehicle.
Vehicle (abstract)	Stores the registration number; defines the fee behaviour common to all vehicles.	Avoids repeating common data in every vehicle class.
TwoWheeler, Car, ElectricCar	Provide the rate of each category; ElectricCar adds charging.	Each category has its own rate, so behaviour is specialised.
ParkingTicket	Records ticket number, vehicle, entry time, exit time and fee.	One object represents one parking session.
ParkingUser (Student, Staff, Visitor)	Holds user type and pass status; decides fee exemption.	Pass rules belong to the user, not to the vehicle.
PricingRule	Holds fee slabs and category rules.	Lets prices change without touching allocation code.
ParkingService	Coordinates the entry and exit workflow.	Keeps application logic apart from data classes.
TicketRepository	Stores and finds active tickets.	Allows the storage method to be replaced (see Task 11).
Chargeable (interface)	Declares the charging capability.	Only some vehicles can charge, so it is a behaviour and not a common property.

A separate ParkingSlot class is optional. The supplied program stores a vehicle number in each slot of a String array, which is enough for the basic operations. A ParkingSlot class would become useful if a slot needs its own attributes, such as being a charging slot.

Business rules. Every valid operation must satisfy the following rules:

- A slot holds at most one vehicle at a time.
- A ticket number is never reused.
- A vehicle that already has an active ticket cannot enter again with the same registration number.
- A slot can be released only if it is occupied, so a slot cannot be released twice.
- The exit time must not be earlier than the entry time, so duration is never negative.

- A student or staff member with a valid monthly pass pays nothing; a visitor pays the full fee.
- EV charging is charged separately from parking.
- If no suitable slot is free, entry is refused and no ticket is issued.

Expected Result

A complete problem analysis with justified candidate classes, business rules and assumptions, which forms the basis for all later tasks.

Alternative Concept and Complexity Analysis

Suppose vehicles were stored as plain Strings in parallel arrays instead of as objects. The following sketch shows how a search and an added attribute would look:

```
String[] regNo     = new String[N];
String[] type      = new String[N];
long[]   entryTime = new long[N];

int find(String reg) {                  // returns index or -1
    for (int i = 0; i < count; i++)
        if (regNo[i].equals(reg)) return i;
    return -1;
}
// To add an attribute (for example 'userType'), a fourth array is created
// and EVERY insert, delete and sort must also update it.
```

Table 3: Vehicle objects compared with parallel arrays

Aspect	Vehicle objects	Parallel arrays
Find by registration	O(n) scan of an array or list; O(1) average if a HashMap keyed by registration is used	O(n) scan of regNo[]; the index is then reused in the other arrays
Add a new attribute	Add one field and accessor to the class	Add another array and update every operation that touches the others
Space per vehicle	O(1): object header plus its fields	O(1): one slot in each array, but capacity N is reserved in advance
Reliability	Related data stays together; type checking by the compiler	Indexes can fall out of step, giving wrong or mixed records

Both designs need O(n) time for a linear search and O(1) space per vehicle, so the difference is not speed but safety and maintainability. Objects keep related state and behaviour together and are easier to extend, so objects are chosen.

6.2 Task 2 – Choose the Data Representation

Purpose

This task decides how each item of data is stored in Java, how floors and slots are arranged in arrays, where type conversion is needed, and how input is read.

Java Concepts Used

Primitive and reference data types, one-dimensional, two-dimensional and jagged arrays, integer division, casting, and the Scanner class.

Implementation Approach

Table 4: Data types chosen for each field

Field	Chosen type	Reason
Floor number, slot number	int	Whole-number indexes that are checked against array bounds.
Vehicle number	String	Contains letters and digits, for example TN01AB1111; arithmetic is never performed on it.
Entry and exit time	LocalDateTime (LocalTime for a time of day)	Supports date-time arithmetic and Duration calculation.
Duration	int minutes (long when taken from Duration)	Whole minutes are enough for billing; Math.toIntExact converts a long safely.
Fee	double in this assignment; BigDecimal in real billing	double matches the supplied listing; BigDecimal avoids binary rounding error in money.
Energy used	int kWh	Matches the chargingFee(int kWh) signature given in the brief.
Slot content	String (null when free)	Matches the supplied ParkingFacility; null clearly represents an empty slot.

Floors and slots. A jagged two-dimensional array is used: `slots[f][s]`, where f is the floor and s the slot on that floor. Each floor is created separately, so floors can have different sizes:

```
slots = new String[slotsPerFloor.length][];      // one row per floor
for (int f = 0; f < slotsPerFloor.length; f++)
    slots[f] = new String[slotsPerFloor[f]];      // each row has its own length
```

A one-dimensional array with the formula index = floor × slotsPerFloor + slot would work only when every floor has the same number of slots. Task 12 introduces floors with different slot counts, so the jagged array is the correct choice from the start.

Type conversion and precision. Fees are based on hours, but durations are measured in minutes, so a conversion is required. Integer division discards the fraction, which gives a wrong fee:

Table 5: Effect of integer division on the fee

Stay	Wrong method: (minutes / 60) × rate	Correct method: ((minutes + 59) / 60) × rate
61 minutes, car (Rs 30/hour)	(61 / 60) = 1, so Rs 30	(61 + 59) / 60 = 2, so Rs 60
130 minutes, car (Rs 30/hour)	(130 / 60) = 2, so Rs 60	(130 + 59) / 60 = 3, so Rs 90

The correct expression computes the ceiling of minutes divided by 60 using integer arithmetic only:

```
int billableHours = (minutes + 59) / 60;     // ceiling for non-negative minutes
double fee = billableHours * ratePerHour();
```

The expression `(int) (minutes / 60.0)` is also unsafe, because a cast to int truncates and does not round up. `Math.ceil(minutes / 60.0)` would be correct but introduces floating-point arithmetic that is not needed.

Input. A Scanner is chosen for an interactive console program because the registration number, vehicle type and times are entered while the program runs. Command-line arguments suit automated runs but are less convenient for repeated entry. Every value is validated before an object is created.

Expected Result

A documented data model in which every field has a justified type, floors are stored in a jagged array, and the billable-hour calculation uses a safe integer ceiling.

Alternative Concept and Complexity Analysis

The brief asks what changes if the slots are stored in a single array with index = floor × slotsPerFloor + slot, or in an ArrayList<String[]>. Let F be the number of floors and S the number of slots per floor.

Table 6: Alternative slot representations

Representation	Time to locate a slot	Unequal floors	Space
Jagged String[][]	O(1) when floor and slot are known; O(F·S) worst case to find the first free slot	Supported directly	O(F·S) references
Single 1-D array	O(1) only if all floors are equal; otherwise a table of floor start offsets is required	Needs an offset array of size F; converting an index back to a floor needs O(log F) binary search	O(F·S), or more if sized for the largest floor
ArrayList<String[]>	O(1) for get(f)[s]; O(F·S) worst case to find a free slot	Supported, since each String[] has its own length	O(F·S) plus a small list overhead

When floors are equal, the single array is the simplest and uses the least memory. When floors differ, it needs extra bookkeeping, so the jagged array or ArrayList is preferred. The jagged array is chosen because it models the floors directly and gives O(1) indexed access.

6.3 Task 3 – Design the Control Logic

Purpose

This task designs the decision and looping logic of the two core operations, allocating a free slot and computing the parking fee, and expresses them in pseudocode before coding.

Java Concepts Used

Nested for loops, if–else and switch statements, and the break, continue and return statements.

Implementation Approach

Allocation. The program searches floor 0 first, then floor 1 and so on, and within each floor from slot 0 upwards. The first free slot is taken, the vehicle number is stored there and the coordinates {floor, slot} are returned immediately. The return statement ends both loops, so no further slots are examined. If every slot has been examined without success, the method returns null. In that case no slot is changed and no ticket is issued, so a full facility is handled without any error.

Use of break, continue and return. A return inside the inner loop is the simplest way to stop as soon as a slot is found. A break would stop only the inner loop and would need a flag to stop the outer loop as well. A continue is useful when a slot must be skipped, for example when it is occupied or not compatible with the vehicle type.

Fee slabs. The brief mentions slabs such as the first hour, each additional hour and a daily maximum. In this assignment every started hour is charged at the vehicle's hourly rate, so the slab logic is: no charge for an exempt user, otherwise hours × rate, limited by the daily maximum if one is set. An if–else chain expresses the exemption and the cap. A switch could select rates by vehicle category, but overriding a rate method in each subclass is more extensible (see Task 7).

Pseudocode – Allocate

```
ALGORITHM Allocate(vehicleNo)
  FOR floor <- 0 TO numberOfFloors - 1
    FOR slot <- 0 TO slotsOnFloor[floor] - 1
      IF slots[floor][slot] is empty THEN
        slots[floor][slot] <- vehicleNo
        RETURN {floor, slot}
      END IF
    END FOR
  END FOR
  RETURN null                       // facility is full
END ALGORITHM
```

Pseudocode – Fee computation

```
ALGORITHM CalculateFee(ratePerHour, minutes, feeExempt, dailyMax)
  IF minutes < 0 THEN RETURN error
  IF feeExempt THEN RETURN 0       // valid monthly pass (student or staff)
  hours <- (minutes + 59) DIV 60   // every started hour is charged
  fee   <- hours * ratePerHour
  IF dailyMax is set AND fee > dailyMax THEN fee <- dailyMax
  RETURN fee
END ALGORITHM
```

A stay of zero minutes gives zero billable hours and therefore a fee of Rs 0. This follows directly from the formula and is recorded as the designed behaviour for the zero-minute boundary case.

Complete the Code – ParkingFacility. The two TODO sections of the supplied class are completed as follows. The rest of the class, including main(), is unchanged.

```
public int[] allocate(String vehicleNo) {
    for (int f = 0; f < slots.length; f++) {
        for (int s = 0; s < slots[f].length; s++) {
            if (slots[f][s] == null) {
                slots[f][s] = vehicleNo;
                return new int[]{f, s};
            }
        }
    }
    return null;
}

public boolean release(int floor, int slot) {
    if (floor < 0 || floor >= slots.length) return false;
    if (slot < 0 || slot >= slots[floor].length) return false;
    if (slots[floor][slot] == null) return false;
    slots[floor][slot] = null;
    return true;
}
```

The order of checks in release() matters: the floor is validated first, because slots[floor] would throw an exception for an invalid floor, and the slot is validated next, before the array is read.

Trace of main(). The facility is created with {2, 1}: floor 0 has two slots and floor 1 has one slot.

Table 7: Expected trace of the ParkingFacility program

Step	Call	Facility state afterwards	Expected output
1	allocate(TN01AB1111)	[0,0] occupied	[0, 0]
2	allocate(TN02CD2222)	[0,0], [0,1] occupied	[0, 1]
3	allocate(TN03EF3333)	[0,0], [0,1], [1,0] occupied	[1, 0]
4	allocate(TN04GH4444)	Unchanged; facility full	null
5	release(0,1), release(0,1), release(5,0)	[0,1] freed by the first call	true false false
6	allocate(TN04GH4444)	[0,1] occupied again	[0, 1]

[Placeholder] The actual output of this program must be captured from the student's own execution and compared with the expected output above (see Chapter 10).

Expected Result

The program prints [0, 0], [0, 1], [1, 0], null, true false false and [0, 1] on separate lines, exactly as given in the brief.

Alternative Concept and Complexity Analysis

Instead of scanning, the facility could keep a count of free slots for each floor and a stack of free slot numbers. Allocation would skip a full floor in O(1) by checking its count, and then

pop a free slot from that floor's stack. Release would push the freed slot back and increase the count.

```
ALGORITHM AllocateFast(vehicleNo)
  FOR floor <- 0 TO numberOfFloors - 1
    IF freeCount[floor] > 0 THEN
      slot <- freeStack[floor].pop()
      slots[floor][slot] <- vehicleNo
      freeCount[floor] <- freeCount[floor] - 1
      RETURN {floor, slot}
  RETURN null
END ALGORITHM
```

Table 8: Scan compared with free-count and stack

Approach	allocate()	release()	Extra space
Scan (implemented)	O(F·S) worst case	O(1)	O(1)
Free-count + stack	O(F) to find a floor, then O(1) to pop	O(1) push and count update	O(F) for counts plus up to O(F·S) for the stacks

A stack returns the most recently freed slot, not the lowest-numbered one, so the order of allocation would differ from the scan. A priority queue (min-heap) would restore lowest-first order at O(log S) per operation. The stack must also never hold the same slot twice, or one slot could be given to two vehicles.

6.4 Task 4 – Design Classes, Objects and Constructors

Purpose

This task defines the fields and methods of the classes found in Task 1, decides how their objects are created, and explains how objects are held in memory.

Java Concepts Used

Fields and methods, default, parameterised and overloaded constructors, the this keyword, constructor chaining, copy constructors, reference variables, stack and heap.

Implementation Approach

Table 9: Fields and methods of the main classes

Class	Main fields	Main methods
Vehicle (abstract)	private final String regNo	getRegNo(), ratePerHour() (abstract), calculateFee(int), equals(), hashCode()
TwoWheeler, Car	Registration inherited from Vehicle	Constructor; ratePerHour() overridden
ElectricCar	Everything inherited from Car	chargingFee(int kWh)
ParkingTicket	ticketId, vehicle, entryTime, exitTime, fee	close(exitTime), getters
ParkingFacility	String[][] slots (jagged)	allocate(), release()
ParkingUser	userId, userType, pass status	isFeeExempt()

Constructors. A default constructor would create an object with safe default values, but a ticket without a vehicle or time has no meaning, so ParkingTicket has no default constructor. Parameterised constructors receive the required values and validate them. Overloaded constructors allow the same object to be created with different information, for example a ticket created with the current time or with a given entry time.

The this keyword and constructor chaining. The keyword this refers to the current object and separates a field from a parameter of the same name. A constructor can call another constructor of the same class with this(...), so validation is written once:

```
class ParkingTicket {
    private final String ticketId;
    private final Vehicle vehicle;
    private final LocalDateTime entryTime;
    private LocalDateTime exitTime;

    ParkingTicket(String id, Vehicle v) {                 // overloaded
        this(id, v, LocalDateTime.now());                 // constructor chaining
    }
    ParkingTicket(String id, Vehicle v, LocalDateTime entry) {
        if (id == null || id.isBlank() || v == null || entry == null)
            throw new IllegalArgumentException("Invalid ticket data");
        this.ticketId = id;
        this.vehicle = v;
        this.entryTime = entry;
    }
}
```

Copy constructor. A copy constructor is useful only in a limited case: printing a duplicate of a lost ticket. A duplicate describes the same parking session, so it keeps the same ticket number and entry time. It must not be stored as a second active ticket, because ticket numbers are unique and a vehicle may have only one active ticket. A new session always receives a new ticket from the ticket number generator, never a copy.

Memory. Local variables and method parameters live in the stack frame of the running method. Objects such as Vehicle and ParkingTicket are created on the heap. A reference variable holds the address of the heap object, so assigning or returning a ticket copies the reference, not the object. An object becomes eligible for garbage collection when no reference can reach it.

Expected Result

Class definitions in which every object is created in a valid state, with validation centralised through constructor chaining.

Alternative Concept and Complexity Analysis

Table 10: Object creation strategies

Strategy	How validity is guaranteed	Time for n tickets	Space
Overloaded constructors with chaining (used)	Strong: the object cannot exist unless validation passes	O(n) overall; O(1) per object	O(n) objects
Default constructor + setters	Weak: the object exists in an incomplete state until every setter has run	O(n) plus the cost of each setter's checks	O(n) objects

Strategy	How validity is guaranteed	Time for n tickets	Space
Static factory method (for example ParkingTicket.issue(v))	Strong: the factory validates, can give a meaningful name and may reuse objects	O(n) overall	O(n) objects, plus a cache if one is used

Creating one ticket takes a constant amount of work for a fixed number of fields, so creating n tickets takes O(n) time and O(n) space in all three approaches. The real difference lies in whether an invalid or half-built object can ever exist, and the constructor approach prevents it.

6.5 Task 5 – Apply static Members and the Object Life Cycle

Purpose

This task finds the data and behaviour that belong to a class as a whole rather than to individual objects, and follows a ticket object through its life.

Java Concepts Used

static variables, static methods, static initialisation blocks, garbage collection, and passing and returning objects by reference value.

Implementation Approach

Static variable. The ticket counter must be static, because it is shared by all tickets and must keep increasing across them. An instance variable would restart for each object and could give the same number to different tickets. Total occupancy, in contrast, belongs to each facility, so it is an instance variable unless a campus-wide total is specifically required.

```
class TicketIdGenerator {
    private static long nextId = 1;
    public static synchronized String next() { return "T" + nextId++; }
}
```

The method is synchronized so that two entry gates calling it at the same time cannot receive the same number. **Static methods and blocks.** A static method suits work that does not depend on a particular object, such as generating the next ticket number or converting minutes to billable hours. A static block can set up constants or configuration once when the class is loaded, but it should not perform fragile work such as reading files.

Life cycle of a ticket. A ticket is created at entry and closed at exit. After exit it may be kept in a history list for auditing; while that list holds a reference, the ticket is still reachable and cannot be garbage collected. Once no reference remains, it becomes eligible for collection. Eligibility does not mean the memory is freed immediately; the JVM decides when to collect.

Passing objects. Java passes the value of a reference. A service method that receives a Vehicle and returns a ParkingTicket works with the same objects as the caller; it can change the state of the object it received, but assigning a new object to its own parameter does not change the caller's variable.

Expected Result

Unique ticket numbers across the whole program, and a clear account of when tickets are retained and released.

Alternative Concept and Complexity Analysis

If the counter were an instance variable of ParkingFacility and the university ran two facilities (north and south gates), each facility would count from 1 independently, and both could issue a ticket numbered T1. Two solutions exist: keep one static counter shared by all facilities, or keep an instance counter in each facility and add a facility code to the number (for example N-1 and S-1), or use a UUID.

Table 11: Static and instance ticket counters

Design	Counter storage for k facilities	Ticket storage for n tickets	Uniqueness
Static counter	O(1)	O(n)	Unique across all facilities
Instance counter per facility	O(k)	O(n)	Unique only if a facility code is added

Counter storage is negligible compared with the O(n) tickets in both designs. The choice is therefore about correctness, not memory.

6.6 Task 6 – Design the Inheritance Hierarchy

Purpose

This task builds an inheritance hierarchy for vehicles so that common data is written once and category-specific data is added only where needed.

Java Concepts Used

Single, multilevel and hierarchical inheritance, the super keyword, abstract classes, interface implementation, and constructor execution order.

Implementation Approach

Vehicle is the superclass. TwoWheeler and Car extend Vehicle directly, which is hierarchical inheritance. ElectricCar extends Car, so the chain Vehicle → Car → ElectricCar is multilevel inheritance. ElectricCar also implements the Chargeable interface. An ElectricCar is therefore a Car, a Vehicle and a Chargeable.

Table 12: What each class inherits and adds

Class	Inherits	Adds or specialises
Vehicle	–	The private registration number, getRegNo(), the abstract ratePerHour(), calculateFee(), equals() and hashCode()
TwoWheeler	Everything from Vehicle	ratePerHour() returning 10
Car	Everything from Vehicle	ratePerHour() returning 30

Class	Inherits	Adds or specialises
ElectricCar	Everything from Car, including the rate of 30	chargingFee(int kWh) at Rs 12 per kWh

Use of super. A subclass constructor calls super(regNo) to pass the registration number to the Vehicle constructor, because the field is private and cannot be set directly by the subclass. The call must be the first statement of the constructor. A subclass method may also call super.method() to reuse the superclass version before adding its own behaviour. Fields stay private, and subclasses read common data through getters.

Users. The same idea can be applied to users: ParkingUser as the superclass with Student, Staff and Visitor as subclasses, where each subclass decides whether it is fee-exempt. This is optional, because user type could also be stored as a field.

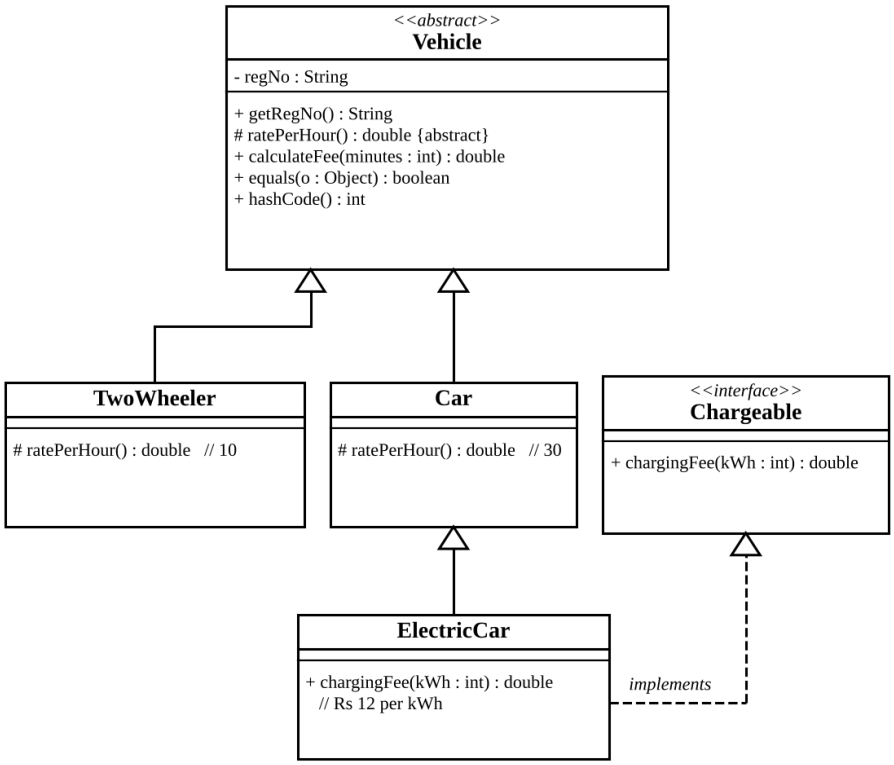


Figure 1: UML class diagram of the vehicle hierarchy

Constructor execution trace for new ElectricCar("TN03C3"). Constructors run from the top of the hierarchy downwards:

Table 13: Constructor execution order

Order	Constructor	Action
1	Object()	Starts implicitly at the top of every Java hierarchy.
2	Vehicle(String regNo)	Stores the registration number TN03C3.
3	Car(String regNo)	Runs after Vehicle completes; Car adds no field of its own.

Order	Constructor	Action
4	ElectricCar(String regNo)	Runs last, finishing the creation of the most specific class.

Expected Result

A justified hierarchy with a UML diagram, correct use of super, and a constructor trace in which the superclass constructor always runs before the subclass constructor.

Alternative Concept and Complexity Analysis

With composition, one Vehicle class would hold a VehicleType object that stores the rate. A new category would then be a new VehicleType object, not a new subclass:

```
class VehicleType { final String name; final double rate; /* ... */ }
class Vehicle {
    private final String regNo;
    private final VehicleType type;           // has-a relationship
    double calculateFee(int minutes) { return ((minutes + 59) / 60) * type.rate; }
}
```

Table 14: Inheritance compared with composition

Aspect	Inheritance (used)	Composition
Adding a new vehicle type	Write a new subclass	Create a new VehicleType object; no new class
Time to compute a fee	O(1)	O(1) after one reference lookup
Space per vehicle	O(1)	O(1), plus one reference to a shared VehicleType
Best suited when	Categories are few and stable	Categories and pricing change independently

Fee computation is a constant-time operation in both designs. Inheritance is simpler for the fixed categories in this assignment; composition becomes attractive if categories are added often.

6.7 Task 7 – Override and Specialise Behaviour

Purpose

This task identifies behaviour that must differ between subclasses and implements it by method overriding.

Java Concepts Used

Method overriding, the @Override annotation, the final keyword, and overriding toString(), equals() and hashCode() of the Object class.

Implementation Approach

What is overridden. In the supplied hierarchy, calculateFee() is written once in Vehicle and depends on the abstract method ratePerHour(). Each subclass overrides ratePerHour() to return its own rate: 10 for TwoWheeler and 30 for Car. ElectricCar inherits the car rate and adds

chargingFee(). This keeps the common fee rule in one place and lets only the rate vary. When a category needs a different fee rule altogether (for example the delivery van in Task 12), calculateFee() itself can be overridden.

Rules of overriding. The @Override annotation makes the compiler check that the method really overrides a superclass method, so a misspelt name is caught at compile time. The overriding method must have the same name and parameter list, the same or a covariant return type, an access level that is not more restrictive, and no broader checked exceptions. For example, ratePerHour() is protected in Vehicle, so a subclass may keep it protected or make it public, but not make it private.

Use of final. A final variable cannot be reassigned, so the registration number and the ticket number are declared final to protect their identity. A final method cannot be overridden and a final class cannot be extended; these should be used only where extension would be harmful, for example a utility class. calculateFee() is deliberately not final, so that special categories can override it.

toString(), equals() and hashCode(). Two Vehicle objects with the same registration number must be treated as equal. The supplied listing therefore defines equality by registration number, and hashCode() uses the same field so that equal objects produce equal hash codes, as the Java contract requires:

```
@Override public boolean equals(Object o) {
    return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
}
@Override public int hashCode() { return regNo.hashCode(); }
@Override public String toString() {
    return getClass().getSimpleName() + "[" + regNo + "]";
}
```

Registration numbers should be converted to a standard form (for example upper case with no spaces) before they are compared, otherwise TN01AB1111 and tn01ab1111 would be treated as different vehicles.

Expected Result

Correct overriding with @Override, justified use of final, and object comparison and printing based on the registration number.

Alternative Concept and Complexity Analysis

A single calculateFee() could use a switch on a type field instead of overriding:

```
double calculateFee(int minutes) {
    int hours = (minutes + 59) / 60;
    switch (type) {                  // type is an enum: TWO_WHEELER, CAR, ...
        case TWO_WHEELER: return hours * 10;
        case CAR:         return hours * 30;
        default: throw new IllegalStateException();
    }
}
```

Table 15: Overriding compared with a switch

Aspect	Overriding (used)	switch on a type field
Time per fee calculation for k types	O(1) by dynamic dispatch	O(1) for an enum switch; up to O(k) for a chain of conditions
Classes changed when a new type is added	One new class; existing classes untouched	The class holding the switch changes, and so does every other class with a similar switch
Risk	Low: the compiler requires the abstract method to be implemented	A forgotten case is a runtime error

6.8 Task 8 – Apply Polymorphism

Purpose

This task demonstrates both compile-time and run-time polymorphism and shows why they remove the need for long chains of type tests.

Java Concepts Used

Method overloading, upcasting, dynamic method dispatch, the instanceof operator and interface-typed access.

Implementation Approach

Compile-time polymorphism. calculateFee() can be overloaded: one version takes the number of minutes and another takes an entry and an exit time. The compiler chooses between them from the number and types of arguments, before the program runs.

```
double calculateFee(int minutes) { ... }
double calculateFee(LocalTime in, LocalTime out) {
    long minutes = java.time.Duration.between(in, out).toMinutes();
    return calculateFee(Math.toIntExact(minutes));
}
```

A LocalTime holds only a time of day, so a stay that crosses midnight (for example 23:30 to 00:30) would give a negative duration. A real system should use LocalDateTime, which carries the date as well.

Run-time polymorphism. An array of type Vehicle can hold a TwoWheeler, a Car and an ElectricCar, because each is a Vehicle (upcasting). When v.calculateFee(130) is called inside a loop, the JVM chooses the method according to the actual object at run time, not the declared type of the reference. The code that processes the vehicles never tests for the type, so no if–else chain is needed, and a new vehicle class works without any change to the loop.

Complete the Code – FeeDemo. The three TODO sections are completed as follows:

```
// TODO 1 – in Vehicle
public double calculateFee(int minutes) {
    int hours = (minutes + 59) / 60;          // ceiling using integers only
    return hours * ratePerHour();
}
// TODO 2 – in Car
Car(String regNo) { super(regNo); }
@Override protected double ratePerHour() { return 30; }
// TODO 3 – in ElectricCar
@Override public double chargingFee(int kWh) { return kWh * 12; }
```

Dispatch trace for the ElectricCar element with v.calculateFee(130):

Table 16: Dynamic dispatch trace

Item	Value
Declared type of v	Vehicle
Actual object type	ElectricCar
Method executed	calculateFee(int) inherited from Vehicle; inside it, ratePerHour() is dispatched to the version in Car
Parking component	3 started hours × Rs 30 = Rs 90
Charging component (instanceof Chargeable)	10 kWh × Rs 12 = Rs 120
Total	Rs 210.0

Expected output of FeeDemo (from the brief):

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Why the last line prints true. The equals() method accepts any Vehicle and compares only the registration number. A Car and an ElectricCar with the registration TN02B2 are therefore equal, although their classes differ. This is acceptable for the system, because the registration number identifies one physical vehicle, and one registration cannot belong to a car and an electric car at the same time. The behaviour does show that two entries with the same number must never be active together, so the system must reject a duplicate registration and standardise the format of registration numbers. If the class had to be part of identity, equals() would compare getClass() as well.

[Placeholder] The actual output must be captured from the student's own execution of FeeDemo (see Chapter 10).

Expected Result

The four output lines above, produced without any type-testing code in the fee loop.

Alternative Concept and Complexity Analysis

The same behaviour could be written with instanceof tests and casts:

```
double fee;
if (v instanceof TwoWheeler)      fee = ((TwoWheeler) v).calculateFee(130);
else if (v instanceof ElectricCar) fee = ((ElectricCar) v).calculateFee(130);
else if (v instanceof Car)         fee = ((Car) v).calculateFee(130);
```

Table 17: Dynamic dispatch compared with instanceof chains

Aspect	Dynamic dispatch (used)	instanceof and casting
Time per call for k vehicle types	O(1)	Up to k tests, so O(k)
Code size	Constant: one call	Grows with every new type
Maintenance	A new subclass needs no change to the caller	Every chain of tests must be found and edited
Ordering risk	None	ElectricCar must be tested before Car, or the wrong branch runs

The test for Chargeable remains appropriate in FeeDemo, because charging is an optional capability, not a category in the hierarchy.

6.9 Task 9 – Introduce Abstraction

Purpose

This task separates what the system does from how it is done, using an abstract class for shared state and an interface for an optional capability.

Java Concepts Used

Abstract classes and abstract methods, interfaces, multiple interface implementation, default methods and static methods in interfaces.

Implementation Approach

Vehicle is an abstract class. All vehicles share state (the registration number) and common behaviour (calculateFee()), but a vehicle on its own has no rate. The method ratePerHour() is therefore abstract: it is declared in Vehicle and must be implemented by each concrete subclass. An abstract class can have fields, constructors and implemented methods, and cannot be instantiated directly.

Chargeable is an interface. Charging is a capability that only some vehicles have, and it does not depend on where a vehicle sits in the hierarchy. An interface describes such a capability without forcing it on every vehicle. The Payable idea suggested in the brief, for fee-paying users, would be a second interface of the same kind.

Why both are used. An abstract class suits a family of related types that share state, and a class can extend only one. An interface suits a capability that unrelated classes may share, and a class can implement many. Using an abstract class for Vehicle and an interface for Chargeable follows this reasoning.

Multiple behaviours. A class can implement several interfaces at once, for example `class ElectricCar extends Car implements Chargeable, Payable`, and thereby gain several independent behaviours.

Default and static methods. The supplied Chargeable declares only chargingFee(int kWh). As a possible extension, a default method could supply behaviour shared by all implementers, and a static method could hold a policy constant, such as the standard charging rate:

```
interface Chargeable {
    double chargingFee(int kWh);
    default boolean supportsCharging() { return true; }   // optional extension
    static double standardRate() { return 12.0; }         // optional extension
}
```

These two members are a design suggestion and are not part of the supplied listing.

Expected Result

An abstract class and an interface, each justified by the kind of relationship it models.

Alternative Concept and Complexity Analysis

If Vehicle were an interface with default methods and Chargeable an abstract class, several things could no longer be expressed.

Table 18: Effect of swapping the abstraction mechanisms

Change	Consequence
Vehicle becomes an interface	An interface cannot hold instance state or a constructor, so the registration number would have to be stored again in every implementing class, or in a separate Registration object.
Chargeable becomes an abstract class	A class can extend only one class. A vehicle that already extends Car could not also extend Chargeable, so multiple capabilities could not be combined.
Time per method call	O(1) in both designs.
Space per object	Depends on where state is stored and how many references exist, not on the choice of abstract class or interface.

6.10 Task 10 – Enforce Encapsulation through Packages and Access Modifiers

Purpose

This task organises the classes into packages and protects the data of each class so that an object cannot be put into an invalid state from outside.

Java Concepts Used

Packages, access modifiers (private, default, protected, public), getters and setters with validation, wrapper classes and autoboxing.

Implementation Approach

Package structure. The following structure follows the example in the brief:

```
src/
  parking/model/     Vehicle, TwoWheeler, Car, ElectricCar,
                     ParkingTicket, ParkingUser, Chargeable
  parking/service/   ParkingFacility, ParkingService, PricingRule,
                     TicketRepository
  parking/app/       Main (console interface using Scanner)
```

Table 19: Access modifiers and the reason for each

Modifier	Used for	Reason
private	Fields of Vehicle, ParkingTicket, ParkingUser and the slots array	Prevents direct change from outside and protects the object's rules.
default (package-private)	Helper classes and methods used only inside one package	Keeps internal helpers hidden from other packages.
protected	ratePerHour() in Vehicle	Lets subclasses override it without making it part of the public interface.
public	Constructors, getters, allocate(), release() and the service methods	The intended operations that other classes may call.

Validation. Constructors and setters check their input. The registration number must be non-null and non-blank; a duration cannot be negative; floor and slot numbers must lie within array bounds; the exit time must not precede the entry time. Values that must never change, such as the ticket number, are final and have no setter.

```
public void setExitTime(LocalDateTime exit) {
    if (exit == null || exit.isBefore(entryTime))
        throw new IllegalArgumentException("Exit time cannot be before entry time");
    this.exitTime = exit;
}
```

Wrapper classes and autoboxing. A collection such as ArrayList<Integer> stores Integer objects. Assigning an int converts it to an Integer (autoboxing) and reading it back converts it to an int (unboxing). Two risks follow. First, comparing Integer objects with == compares references, which can give true for small values (typically −128 to 127, which are cached) and false for larger ones, so equals() must be used. Second, unboxing a null value throws a NullPointerException.

Expected Result

A clear package structure, a justified access level for every member, and validation that keeps every object valid.

Alternative Concept and Complexity Analysis

If fields were public and callers validated the data, any class could assign a negative duration, a null registration or a slot outside the array, and the object would silently become invalid. Each caller would have to repeat the same checks, and one forgotten check would be a hidden bug.

Table 20: Encapsulation compared with public fields

Aspect	Private fields with validation (used)	Public fields
Time cost of validation per update	O(1)	O(1) per check
Places where the same check is written	One (constructor or setter)	One per calling site, say m sites
Effect of changing a rule	Edit one place	Edit and re-test m places
Risk of an invalid object	Low	High

6.11 Task 11 – Implement, Test and Defend the Design

Purpose

This task implements the whole system, tests it with normal, boundary and invalid cases, checks a hand-calculated scenario and evaluates how well the design follows the object-oriented principles.

Java Concepts Used

All concepts of Tasks 1 to 10, together with collections for storing active tickets.

Implementation Approach

The tests are grouped as the brief requires. **Normal operation:** vehicles of different types enter and leave. **Boundary cases:** the last free slot, a zero-minute stay and a stay exactly on a billing boundary (60 minutes), together with one minute over it (61 minutes). **Invalid cases:** an unknown vehicle type, a duplicate registration number and an invalid slot number. The complete test table, with expected results and a column for actual results, is given in Chapter 9.

Hand-calculated daily collection. A small scenario is calculated by hand first. The program's total for the same inputs must match it. The rates are those of the brief, and the user categories are chosen by the student for this example.

Table 21: Hand-calculated collection for one day

Vehicle	Parking time	Billable hours	Calculation	Fee (Rs)
Two-wheeler (visitor)	90 min	2	2 × 10	20
Car (visitor)	185 min	4	4 × 30	120
Electric car (visitor)	60 min, 8 kWh	1	1 × 30 + 8 × 12	126
Car (visitor)	45 min	1	1 × 30	30
Car (student, valid monthly pass)	240 min	–	Exempt	0
Total collection				**296**

[Placeholder] Program total from the student's own run: Rs [____]. It must equal Rs 296 for the evaluation to be satisfied.

Design evaluation.

- **Encapsulation:** fields are private and changed only through validated constructors and setters.
- **Inheritance:** all vehicles reuse the registration number and the common fee rule.
- **Polymorphism:** the rate is chosen by the run-time type of the object, with no type-testing chains.
- **Abstraction:** Vehicle hides the rate of each category, and Chargeable expresses the EV capability.
- **Limitation:** a deep inheritance hierarchy becomes rigid when vehicle category and pricing policy change independently, because each combination might need a new subclass. A separate PricingRule abstraction, introduced in Task 12, removes this weakness.

Expected Result

A working implementation whose test results match the expected results and whose daily collection equals Rs 296. All claims about the program running must be supported by the student's own captured output.

Alternative Concept and Complexity Analysis

The brief stores active tickets in an array. On exit, the ticket for a registration number must be found. The alternatives are an ArrayList<ParkingTicket> and a HashMap<String, ParkingTicket> keyed by registration number:

```
// HashMap version of exit processing
ParkingTicket t = activeTickets.remove(regNo);     // O(1) average
if (t == null) return reject("No active ticket");
t.setExitTime(exitTime);
facility.release(t.getFloor(), t.getSlot());
```

Table 22: Active-ticket storage structures

Structure	Time to find a ticket on exit	Space for n parked vehicles	Notes
Array	O(n) scan	O(n); fixed capacity	Simple, but the capacity is fixed in advance.
ArrayList<ParkingTicket>	O(n) scan	O(n)	Resizes automatically; search is still linear.
HashMap<String, ParkingTicket>	O(1) average, O(n) worst case	O(n) plus hash table overhead	Fastest lookup; keys must be standardised and unique.

A HashMap keyed by registration number is recommended for active tickets, with a separate history list for closed tickets. It also rejects a duplicate registration naturally, since a key can appear only once.

The brief asks for measured evidence where possible. The table below is reserved for the student's own timing measurements; no values are claimed here.

Table 23: Measured lookup time (to be filled from the student's own run)

n parked vehicles	ArrayList lookup time	HashMap lookup time
1,000	[Placeholder]	[Placeholder]
10,000	[Placeholder]	[Placeholder]
100,000	[Placeholder]	[Placeholder]

6.12 Task 12 – Re-evaluate and Adapt the Design

Purpose

This design challenge tests whether the original design survives changed requirements. It requires an impact analysis, an evaluation of the original design, an adaptation or redesign, and an evidence-based defence.

Changed Requirement

The university introduces two vehicle categories: e-bikes, charged a flat fee, and delivery vans, charged at double the car rate from 9 AM to 5 PM. Visitors may now buy a one-day pass. The facility is extended from 3 to 5 floors, each with a different number of slots.

A. Impact Analysis

Table 24: Impact of the changed requirements

Area	Effect of the change
Vehicle hierarchy	Two new categories, EBike and DeliveryVan, must be added.
Fee calculation	The van's fee depends on the time of day, so a duration in minutes alone is no longer enough; entry and exit times, or a pricing rule that receives them, are required.
Pass model	A one-day pass is valid only on one date, so a Pass abstraction with a validity check is needed next to the monthly pass.
Floor layout	Floors have different slot counts, so a rectangular array would waste space or be wrong. The jagged array already suits this, and the facility is created from an int[] of slot counts.
Service logic	Entry and exit must ask the pass and the pricing rule, so ParkingService is affected.

Classes that would have to be modified, not just extended, are those containing a switch over vehicle types or hard-coded rates, and the entry–exit service that currently computes fees directly. Classes that depend only on stable abstractions (Vehicle, ParkingTicket, Chargeable) can stay as they are.

B. Evaluation of the Original Design

New vehicle types. Because fee behaviour is polymorphic, a new vehicle type can usually be added by writing a new subclass, without editing existing subclasses. This works well for the e-bike. **Pricing rules.** The van's time-of-day rule and the pass rule cut across categories. If such

rules are written inside several Vehicle subclasses or inside the service, they become scattered, and a rule change touches many classes. **Existing tested classes.** Allocation, ticket and slot classes do not depend on pricing, so they should remain unchanged and keep their tests. The weakness found is therefore the location of pricing and pass rules, not the vehicle hierarchy itself.

C. Adaptation or Redesign

Decision: MODIFY the design; do not redesign it. Vehicle, ParkingFacility, ParkingTicket and Chargeable are kept. New vehicle subclasses are added, pricing is moved behind a PricingRule interface, and a Pass interface is introduced. The jagged array for floors is kept.

```
Vehicle (abstract)
  |-- TwoWheeler
  |-- Car --- ElectricCar (implements Chargeable)
  |     '-- DeliveryVan            (new)
  '-- EBike                        (new)

PricingRule (interface)  <--  StandardHourlyRule, FlatFeeRule,
                              DeliveryVanTimeWindowRule        (new)
Pass (interface)         <--  MonthlyPass, OneDayPass          (new)
ParkingFacility  -->  String[][] slots (jagged), created from int[] slotsPerFloor
```

Assumptions for items the brief does not state (to be confirmed with the course faculty):

- E-bike flat fee: the amount is not given. It is written as FLAT_FEE = [Rs ____ – to be decided by the student].
- Delivery van: the rate is double the car rate (Rs 60 per hour) for each started hour whose start falls between 9 AM and 5 PM, and the normal car rate (Rs 30 per hour) otherwise. This is the student's interpretation of the brief.
- One-day pass: a valid pass covers the parking fee on its date of validity only; on any other date the visitor pays the normal fee. The brief does not state the fee for a pass holder, so this must be confirmed.
- Five-floor facility: slot counts are illustrative only, for example {40, 55, 60, 45, 30} (total 230). The brief gives no counts.

Proposed implementation of the new categories and the one-day pass. The code below is a design sketch of the approach and must be replaced by, or checked against, the code in the student's repository.

```
class EBike extends Vehicle {
    EBike(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return FLAT_FEE; }
    @Override public double calculateFee(int minutes) {
        return FLAT_FEE;                        // flat fee, charged once
    }
}

class DeliveryVan extends Car {
    DeliveryVan(String regNo) { super(regNo); }
    public double calculateFee(LocalDateTime in, LocalDateTime out) {
        return DeliveryVanTimeWindowRule.calculate(in, out, ratePerHour());
    }
}

interface Pass { boolean isValid(LocalDateTime at); }
class OneDayPass implements Pass {
    private final LocalDate date;
    OneDayPass(LocalDate date) { this.date = date; }
    public boolean isValid(LocalDateTime at) {
        return date.equals(at.toLocalDate());
    }
}
```

Algorithm for the van's time-window fee (proposed):

```
ALGORITHM VanFee(entry, exit, carRate)
  minutes <- minutes between entry and exit
  IF minutes < 0 THEN RETURN error
  hours <- (minutes + 59) DIV 60
  fee <- 0
  FOR i <- 0 TO hours - 1
    blockStart <- entry + i hours
    IF time of blockStart >= 09:00 AND time of blockStart < 17:00 THEN
      fee <- fee + 2 * carRate
    ELSE
      fee <- fee + carRate
    END IF
  END FOR
  RETURN fee
END ALGORITHM
```

D. Evidence-Based Defence

Table 25: Evidence supporting the MODIFY decision

Evidence	Defence of the decision to modify
Number of existing classes changed	Expected: only the fee-computation entry point (Vehicle or ParkingService) and the service's pass check. Allocation, ticket identity and ParkingFacility stay stable. [Placeholder – confirm the actual number from the repository's commit differences]
Ease of adding a further vehicle type	Add one subclass and, if its pricing is special, one PricingRule; no switch statement and no change to allocation.
Test results for the new categories	See tests T23 to T29 in Chapter 9. [Placeholder – actual results to be filled]

Evidence	Defence of the decision to modify
Trade-offs	More classes and interfaces at the start, but pricing and pass policies are independently testable and a change stays local. A full redesign would discard tested code for no clear gain.

Expected Result

A modified design that keeps the stable core, adds the new categories and pass through extension, and moves variable pricing rules behind one abstraction.

7. System Design

7.1 Classes and Objects

The program is built from a small number of cooperating classes, identified in Task 1 and refined in Tasks 4 to 10. Vehicle objects represent the vehicles, a ParkingTicket object represents one parking session, a ParkingFacility object owns the floors and slots, and a ParkingUser object carries the pass information. A ParkingService object coordinates them, a TicketRepository stores active tickets, and a PricingRule object supplies the fee.

7.2 Relationships between Classes

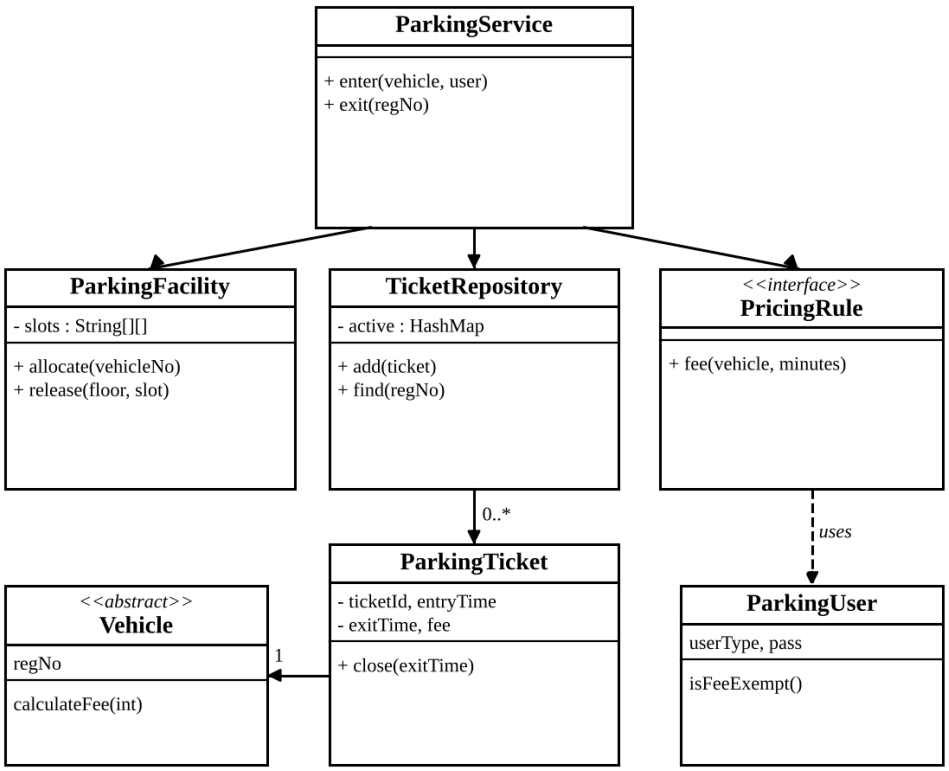


Figure 2: Class relationships of the proposed design

- **ParkingService uses** ParkingFacility (to allocate and release slots), TicketRepository (to store and find active tickets) and PricingRule (to compute fees).
- **TicketRepository holds** zero or more active ParkingTicket objects.
- **ParkingTicket refers to** exactly one Vehicle.
- **PricingRule uses** vehicle and user information, including whether the user is fee-exempt.
- **Vehicle** is the root of the inheritance hierarchy shown in Figure 1; ElectricCar also implements Chargeable.

This diagram is a design proposal based on the responsibilities identified in Task 1. The exact class and method names in the student's repository may differ and should be compared with it before submission.

7.3 Overall Program Structure

The classes are grouped into three packages. The model package holds the data classes (Vehicle and its subclasses, ParkingTicket, ParkingUser, Chargeable). The service package holds the logic that works on the model (ParkingFacility, ParkingService, PricingRule, TicketRepository). The app package holds the console interface that reads input with Scanner and prints results. Keeping the packages separate means that the input method or the storage method can be changed without altering the model.

7.4 Parking Allocation Process

When a vehicle arrives, the service first checks that the registration number is valid and has no active ticket. It then asks the facility for a slot. The facility scans floor by floor and slot by slot, and the first free slot is filled with the vehicle number and returned as {floor, slot}. The service then creates a ticket with a new unique number and the current time and stores it in the repository. If the facility returns null, the entry is refused and no ticket is created.

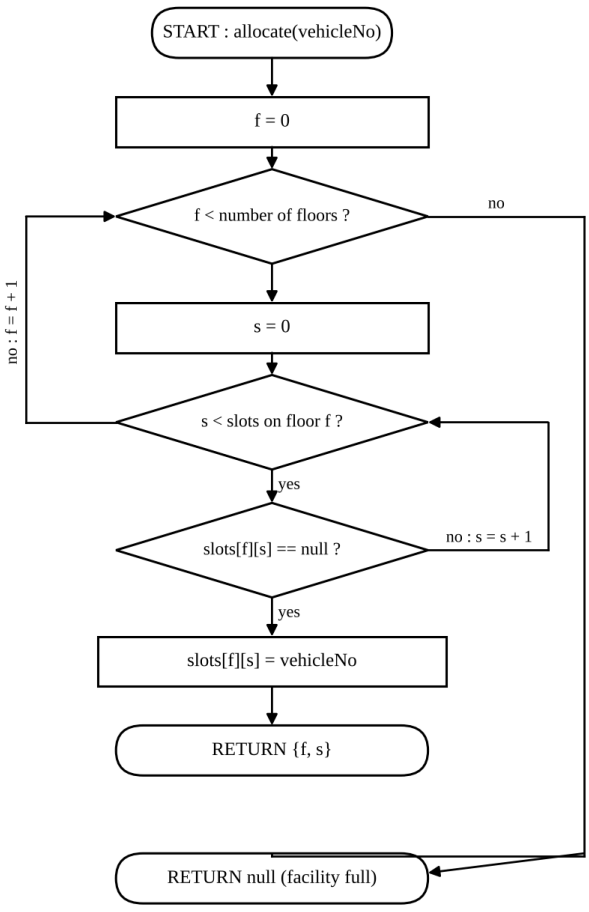


Figure 3: Flowchart of slot allocation

7.5 Vehicle Release Process

When a vehicle leaves, the service looks up its active ticket by registration number. If none exists, the exit is rejected. The parking duration is calculated from the entry and exit times and

rejected if negative. The fee is computed from the vehicle category, the duration and the user's pass status. The slot is then released, and the ticket is closed, removed from the active tickets and kept in history. A slot that is already free or invalid cannot be released, so a second exit with the same ticket fails safely.

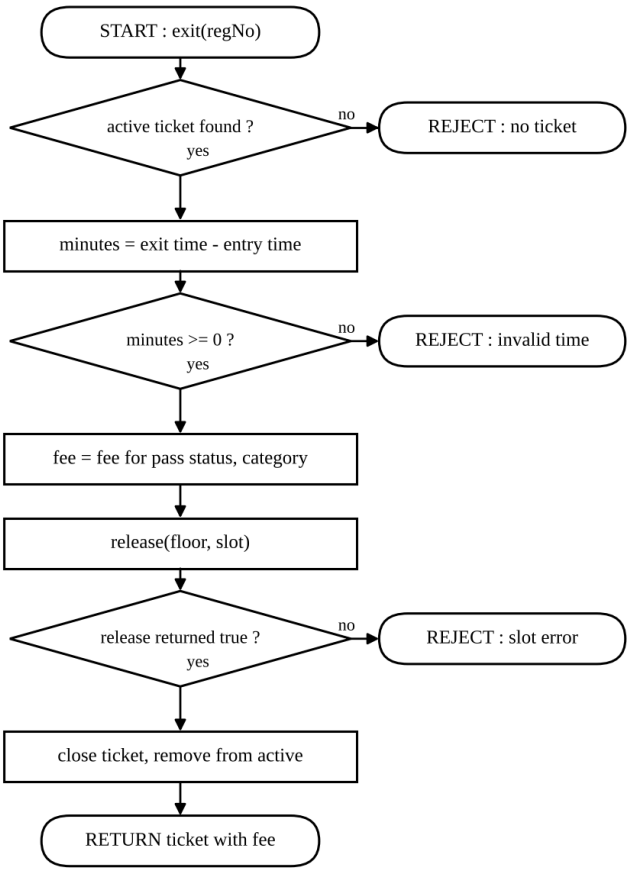


Figure 4: Flowchart of vehicle exit and slot release

8. Algorithms and Implementation

The important algorithms are explained below in simple terms. Where the pseudocode has already been given in a task, the section refers to it instead of repeating it.

8.1 Slot Allocation

The allocation algorithm is a linear search over a jagged array. It stops at the first free slot and returns its coordinates, and returns null when the search ends without success. The pseudocode is in Section 6.3 and the flowchart is Figure 3. In the worst case the facility is full and every slot is examined, which takes time proportional to the total number of slots.

8.2 Slot Release and Vehicle Exit

Release validates the floor, then the slot, then checks that the slot is occupied, and only then empties it. The exit procedure built around it is:

```
ALGORITHM Exit(regNo, exitTime)
  ticket <- activeTickets.get(regNo)
  IF ticket is null THEN REJECT "no active ticket"
  minutes <- minutes between ticket.entryTime and exitTime
  IF minutes < 0 THEN REJECT "invalid time"
  fee <- CalculateFee(ticket.vehicle, minutes, user)
  IF NOT facility.release(ticket.floor, ticket.slot) THEN REJECT "slot error"
  ticket.close(exitTime, fee)
  activeTickets.remove(regNo)
  history.add(ticket)
  RETURN ticket
END ALGORITHM
```

8.3 Fee Computation

The fee is zero for a fee-exempt user. Otherwise the billable hours are the ceiling of minutes divided by 60, calculated with the integer expression (minutes + 59) / 60, and the fee is hours multiplied by the hourly rate, limited by the daily maximum if one is set. The pseudocode is in Section 6.3. EV charging is added separately as kWh multiplied by Rs 12.

8.4 Active Ticket Lookup

Active tickets are stored in a HashMap whose key is the standardised registration number. Lookup, insertion and removal take constant time on average. Because a key can occur only once, an attempt to enter again with the same number is detected immediately.

8.5 Time-Window Fee for Delivery Vans

The van fee is computed one started hour at a time: each hour is charged at double the car rate if it starts between 9 AM and 5 PM, and at the normal car rate otherwise. The pseudocode is in Section 6.12.

8.6 Java Concepts Used and Where

Table 26: Java concepts and where they appear

Java concept	Where it is used	Task
Jagged arrays	Floors and slots in ParkingFacility	2, 3, 12
Control statements (for, if–else, return, continue)	Slot search, fee slabs, validation	3
Classes, objects, constructors, this	ParkingTicket and the vehicle classes	4
static members	Ticket number generator	5
Inheritance and super	Vehicle, TwoWheeler, Car, ElectricCar	6
Overriding, final, equals and hashCode	ratePerHour(), registration number identity	7
Polymorphism (overloading and overriding)	calculateFee() overloads; Vehicle[] loop in FeeDemo	8
Abstract class and interface	Vehicle and Chargeable	9
Packages, access modifiers, wrapper classes	model, service and app packages; private fields	10
Collections (ArrayList, HashMap)	Active ticket storage	11
Exception handling	IllegalArgumentException on invalid constructor or setter data	4, 10

Only concepts that appear in the assignment brief or in its program listings are listed above.

9. Testing and Validation

The tests cover normal operation, boundary cases and invalid cases, using the facility of the supplied main() and the rates given in the brief. The Expected Result column is derived from the brief or by hand calculation. The Actual Result column is intentionally left as a placeholder, because no execution evidence has been provided; no test is claimed as executed.

9.1 Allocation and Release (Task 3)

Table 27: Allocation and release tests

ID	Feature	Input or scenario	Expected result	Actual result
T01	Allocate	1st vehicle, facility {2,1}	[0, 0]	[to be filled]
T02	Allocate	2nd vehicle	[0, 1]	[to be filled]
T03	Allocate	3rd vehicle (floor 0 full)	[1, 0]	[to be filled]
T04	Facility full	4th vehicle	null; no ticket	[to be filled]
T05	Release	release(0, 1)	true	[to be filled]
T06	Repeated release	release(0, 1) again	false	[to be filled]
T07	Invalid floor	release(5, 0)	false; state unchanged	[to be filled]
T08	Reuse of slot	allocate after release	[0, 1]	[to be filled]
T09	Last free slot	allocate when one slot remains	Succeeds at that slot	[to be filled]
T10	Invalid slot	release(0, -1) or slot index too large	false; state unchanged	[to be filled]

9.2 Fee Calculation and Polymorphism (Tasks 2, 7, 8)

Table 28: Fee calculation tests

ID	Feature	Input or scenario	Expected result	Actual result
T11	Two-wheeler fee	130 minutes	3 hours; Rs 30.0	[to be filled]
T12	Car fee	130 minutes	3 hours; Rs 90.0	[to be filled]
T13	EV fee	130 minutes parking + 10 kWh	Rs 90 + Rs 120 = Rs 210.0	[to be filled]
T14	Equality	Car and ElectricCar, same registration	true	[to be filled]
T15	Zero-minute stay	0 minutes	0 hours; Rs 0	[to be filled]
T16	Slab boundary	60 minutes, car	1 hour; Rs 30	[to be filled]
T17	Just over boundary	61 minutes, car	2 hours; Rs 60	[to be filled]

ID	Feature	Input or scenario	Expected result	Actual result
T18	Negative duration	exit time before entry time	Rejected; no fee	[to be filled]
T19	Monthly pass	student with valid pass, any duration	Rs 0	[to be filled]

9.3 Invalid Entries and Scenario (Task 11)

Table 29: Invalid-entry and scenario tests

ID	Feature	Input or scenario	Expected result	Actual result
T20	Unknown vehicle type	unrecognised category input	Validation error; no allocation	[to be filled]
T21	Duplicate registration	same number with an active ticket	Entry rejected	[to be filled]
T22	Daily collection	scenario of Table 7 (five vehicles)	Rs 296 (hand calculation)	[to be filled]

9.4 Changed Requirements (Task 12)

Tests T23 to T29 apply under the assumptions stated in Section 6.12.

Table 30: Tests for the changed requirements

ID	Feature	Input or scenario	Expected result	Actual result
T23	E-bike	any duration	One flat fee [amount to be set]	[to be filled]
T24	Van inside window	10:00 to 12:00	2 h × Rs 60 = Rs 120	[to be filled]
T25	Van outside window	18:00 to 20:00	2 h × Rs 30 = Rs 60	[to be filled]
T26	Van across window start	08:00 to 10:00	Rs 30 + Rs 60 = Rs 90	[to be filled]
T27	One-day pass, valid date	pass date equals entry date	Pass valid	[to be filled]
T28	One-day pass, next day	entry date after pass date	Pass invalid; normal tariff	[to be filled]
T29	Five-floor facility	five unequal floors	Each floor has its configured capacity; total is the sum	[to be filled]

10. Results and Discussion

10.1 Expected Functionality and Outcomes

When the completed ParkingFacility program is run, it is expected to print the following lines, which come from the brief:

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

When FeeDemo is run, it is expected to print:

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

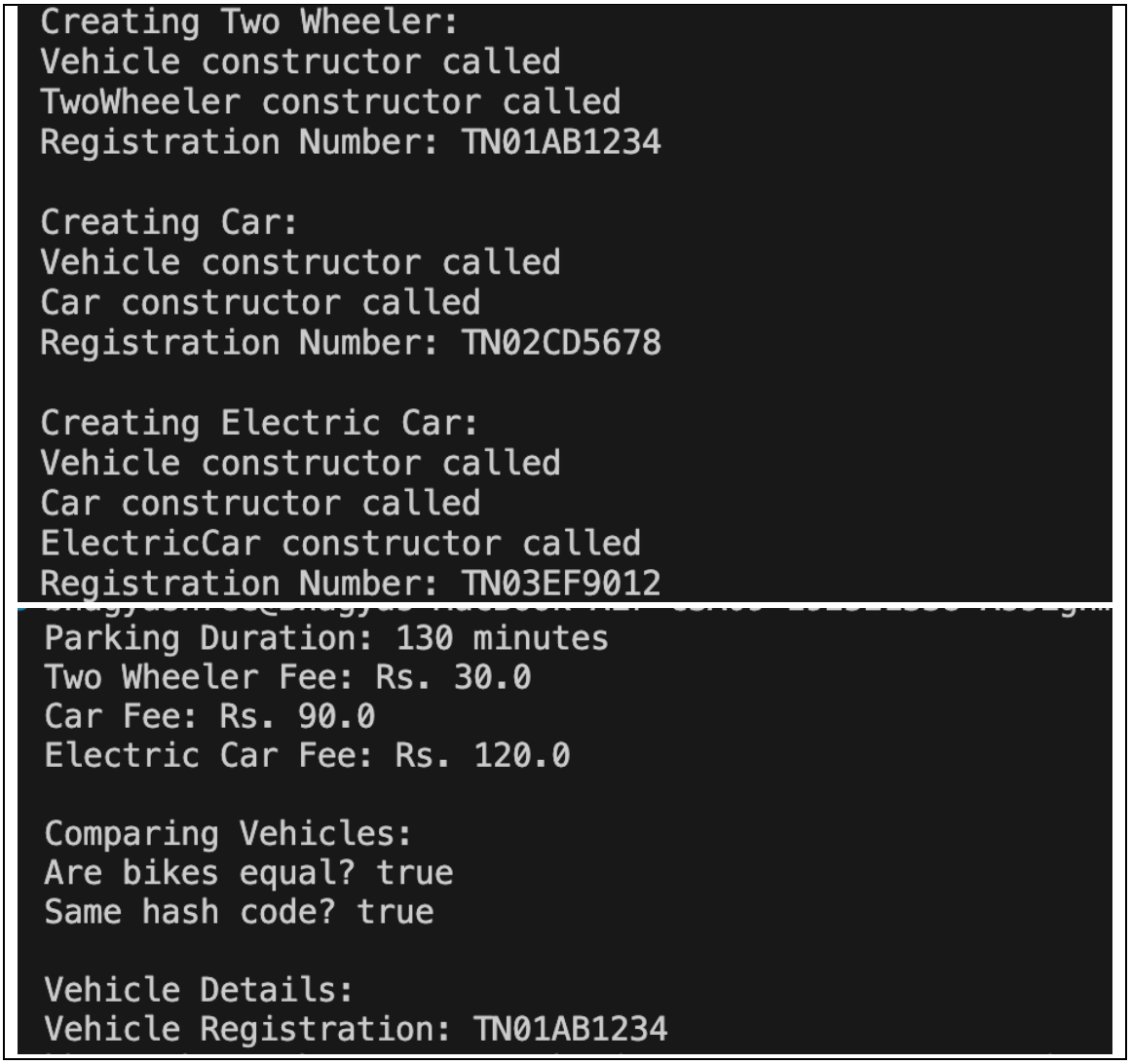
Beyond these programs, the complete system is expected to allocate slots in floor-then-slot order, charge started hours in full, treat valid monthly pass holders as free of charge, reject duplicate entries and invalid releases, and produce a daily total of Rs 296 for the hand-calculated scenario of Task 11.

10.2 Screenshots of Actual Program Execution

The boxes below mark where screenshots of the student's own execution must be inserted. No terminal output has been created for this report.



Figure 5: Task 3 – ParkingFacility program output (compile and run)


Creating Two Wheeler:
Vehicle constructor called
TwoWheeler constructor called
Registration Number: TN01AB1234
Creating Car:
Vehicle constructor called
Car constructor called
Registration Number: TN02CD5678
Creating Electric Car:
Vehicle constructor called
Car constructor called
ElectricCar constructor called
Registration Number: TN03EF9012
Parking Duration: 130 minutes
Two Wheeler Fee: Rs. 30.0
Car Fee: Rs. 90.0
Electric Car Fee: Rs. 120.0
Comparing Vehicles:
Are bikes equal? true
Same hash code? true
Vehicle Details:
Vehicle Registration: TN01AB1234

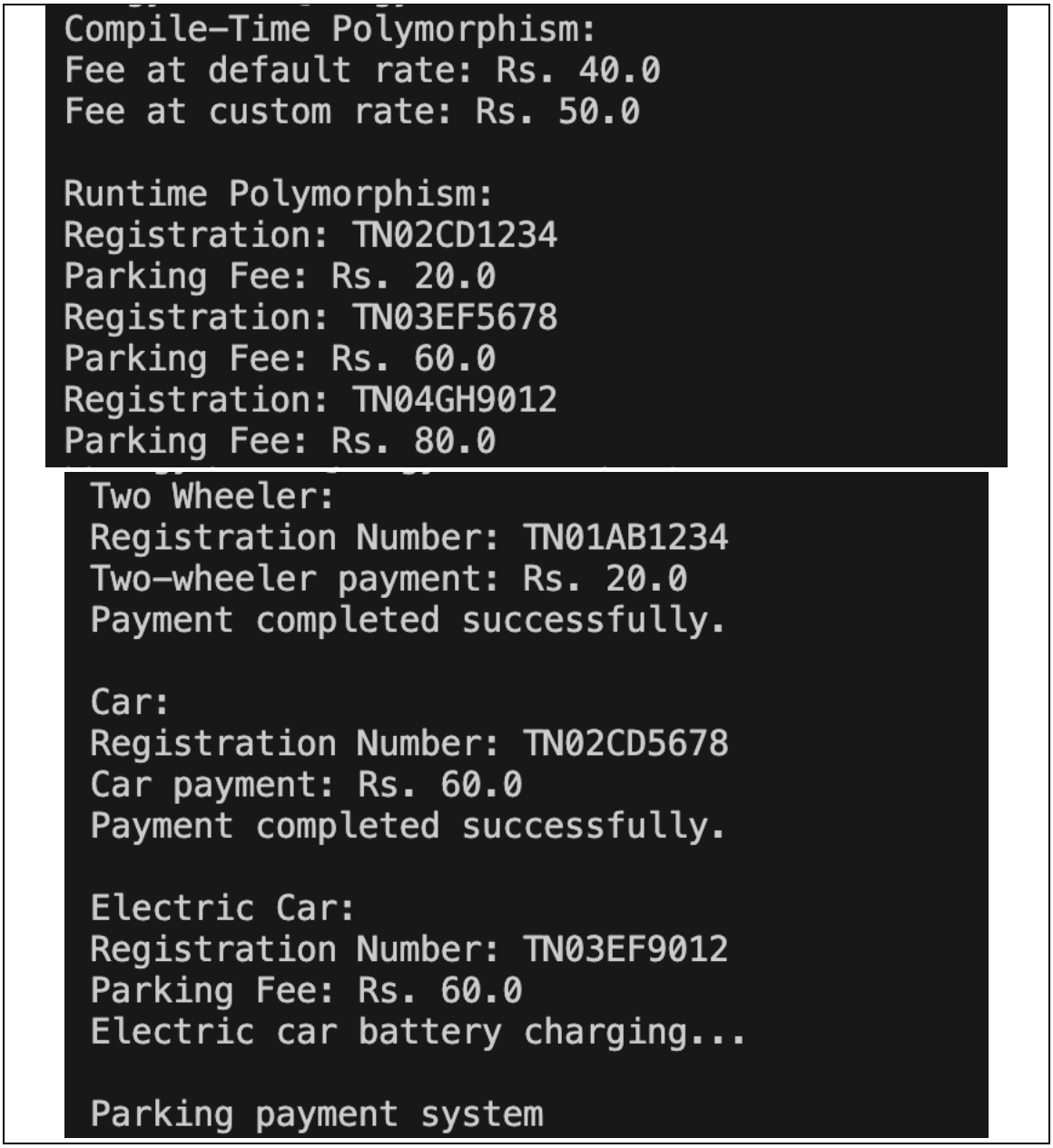


Figure 6: Tasks 6 to 9 – FeeDemo program output

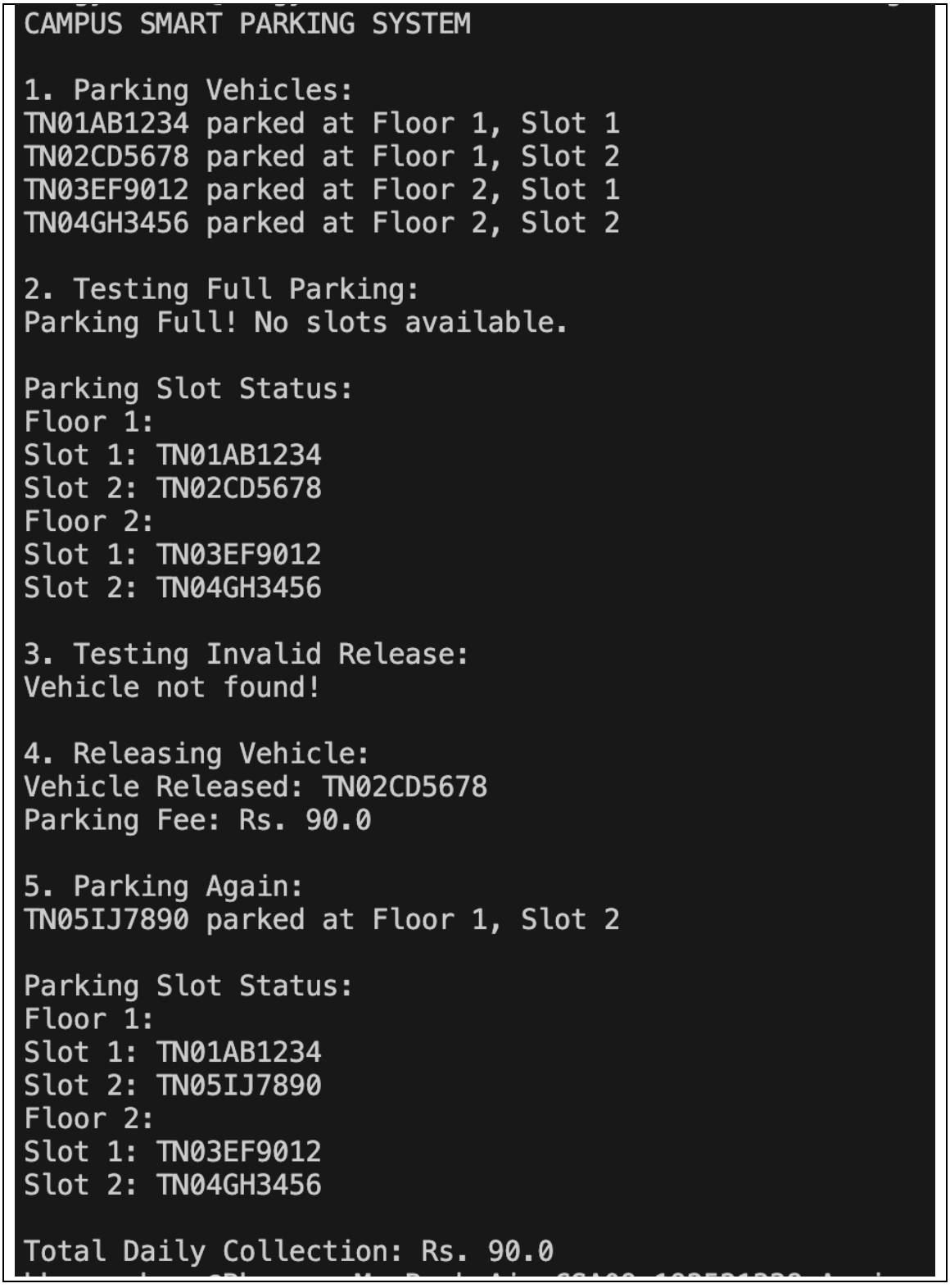


Figure 7: Task 11 – test run covering normal, boundary and invalid cases

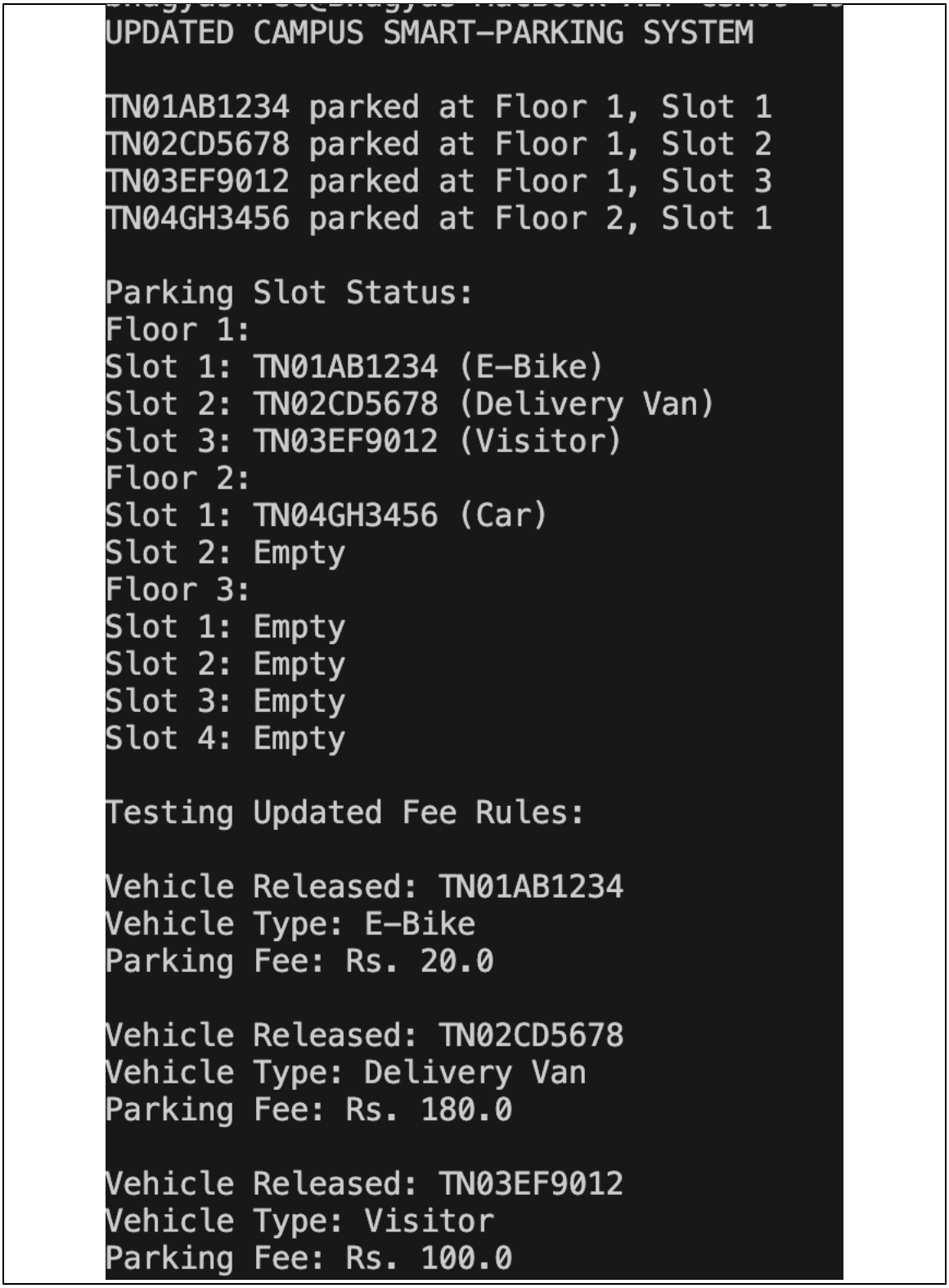


Figure 9: Task 12 – tests for e-bike, delivery van, one-day pass and five-floor facility

10.3 Discussion

The design separates concerns clearly. Slot management is isolated in ParkingFacility, fee rules depend on the vehicle category through overriding, optional capabilities are expressed through

an interface, and data is protected by private fields. These choices follow the evaluation criteria of the assignment and are expected to make the system straightforward to test.

The main risk identified is the placement of pricing rules. The original hierarchy handles simple per-category rates well, but time-dependent and pass-dependent rules cut across categories. Moving them behind a PricingRule abstraction (Task 12) addresses this risk without disturbing the tested allocation and ticket classes. Whether this expectation holds in practice must be confirmed by the student's own test results.

11. Complexity Analysis

The analysis covers only algorithms that appear in the assignment. Notation: F is the number of floors, S the number of slots per floor (so the total is F·S slots, and the facility may have unequal floors, in which case F·S is the total), n the number of parked vehicles or tickets, k the number of vehicle types, and h the number of billable hours. The assumptions are that the facility is scanned sequentially, that hash functions distribute keys well, and that arithmetic on integers takes constant time.

Table 31: Summary of time and space complexity

Algorithm or operation	Time	Extra space	Assumption or note
allocate() by scanning	O(F·S) worst case	O(1)	Worst case: facility full or the free slot is last
release()	O(1)	O(1)	Direct array access after bounds checks
allocate() with free count and stack	O(F) per call	O(F) + O(F·S) for stacks	Stack must not hold duplicates
Fee calculation (rate × hours)	O(1)	O(1)	Integer arithmetic
Dynamic dispatch of calculateFee()	O(1)	O(1)	Versus up to O(k) for instanceof chains
Find vehicle in parallel arrays or ArrayList	O(n)	O(1)	Linear scan by registration number
Find ticket in HashMap	O(1) average; O(n) worst case	O(n) for the table	Standardised, unique keys
Create one ParkingTicket	O(1)	O(1)	Fixed number of fields; n tickets take O(n)
Create the jagged slot array	O(F·S)	O(F·S)	One reference per slot, initially null
Van fee (hour by hour)	O(h)	O(1)	One test per started hour

For a facility of 5 floors and 100 slots per floor, the scan examines at most 500 slots, which is negligible for a single entry. The choice of a faster structure therefore matters mainly for much larger facilities or for the measurement-based comparison requested in the assignment, where the student's own timings should be reported (Table 12).

12. Limitations

- Allocation by scanning takes O(F·S) time in the worst case and ignores the vehicle type, so an EV is not steered to a charging slot unless the slot model is extended.
- Fees are held in double values for simplicity. Real billing would need BigDecimal and a stated rounding policy.
- A deep inheritance hierarchy becomes rigid when category and pricing vary independently, which is the reason for the PricingRule abstraction in Task 12.
- Times are handled in one local time zone and in whole minutes, with no handling of daylight-saving changes.
- Storage is in memory only, so all tickets are lost when the program ends.
- Several details are not stated in the brief (daily maximum, e-bike flat fee, van tariff outside the window, one-day pass fee). Assumptions were made and must be confirmed with the faculty.
- This report contains expected results only; actual outputs, screenshots and the measured timings are placeholders until the student's own execution is recorded.

13. Future Enhancements

- Store tickets and history in a database or file so that they survive a restart.
- Add a graphical or web interface for gate operators in place of the console.
- Introduce a ParkingSlot class with type attributes (for example charging or two-wheeler-only slots) and allocate by compatibility.
- Use the free-count and stack structure, or a min-heap, for faster allocation in very large facilities.
- Support several facilities (for example north and south gates) with facility-coded ticket numbers.
- Read rates, slot counts and pass rules from a configuration file so that they can change without recompiling.
- Add reports for daily collection and occupancy, and automated unit tests for all pricing rules.

14. Conclusion

This report has analysed, designed and planned the validation of an object-oriented Campus Smart-Parking System across all twelve tasks of the assignment. A jagged array represents floors of unequal size, a safe integer ceiling prevents fee errors, and a vehicle hierarchy with an abstract Vehicle class and a Chargeable interface provides category-specific behaviour through overriding and polymorphism. Private fields, validated constructors and a package structure protect the state of each object.

When the requirements changed in Task 12, the design was modified rather than redesigned. The stable core (facility, ticket and vehicle abstraction) was kept, while new vehicle classes, a Pass abstraction and a PricingRule abstraction absorbed the change. This decision rests on the

analysis in this report and must be confirmed by the student's own test results and repository evidence before submission.